

Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- İlişkisel Cebir (Relational Algebra) Temel Operatörleri
 - İlişkisel cebir, SQL'in altyapısını oluşturan makineye yakın bir sorgulama dili
 - SQL son kullanıcıya yönelik, ilişkisel cebir ise makineye yakın ve verinin nasıl erişileceğini detaylı ifade eder
 - İlişkisel cebir anlamak, SQL yazmayı kolaylaştırır
 - SQL standart tanımlaması standarttır ancak gerçekleştirmesi farklı veri tabanlarında farklılık gösterebilir
- Tek Tablo Operatörleri
 - Select (σ) - Yatay Seçim
 - * Tablodaki bazı satırları bir yükleme (predicate) göre seçmek
 - * Yükleme, AND, OR, NOT terimlerinden oluşan bir ifade dizisi
 - * Matematiksel notasyon: $\sigma_p(R)$ — R tablosundan p yüklemine sağlayan tapılları seç
 - Project (π) - Dikey Seçim
 - * Tablodaki istenilen sütunları (nitelikleri) seçmek
 - * Yükleme yoktur, sadece nitelik isimleri belirtilir
 - * İlişkisel cebir set esaslı olduğu için tekrarlar (duplicate) otomatik çıkarılır
 - * Gerçek veri tabanında project yapılırsa tekrarlar çıkarılmaz, set teorisi ile fark oluşur
 - Rename (ρ) - İsim Değişikliği
 - * Nitelik veya tablo isimlerini değiştirmek için kullanılır
 - * Özellikle Natural Join'da aynı isimli nitelik haline getirmek için kullanılır
 - Extend - Şema Genişletme/Değiştirme
 - * Tabloya yeni nitelik eklemek veya mevcut niteliği hesaplanmış değerle değiştirmek
 - * Örnek: Student tablosuna "College = YTÜ" niteliği eklemek
 - Kümeleme Fonksiyonları (Aggregate Functions)
 - * Süslü F notasyonu ile ifade edilir
 - * Fonksiyonun sağ tarafına yazılır: count, sum, avg, min, max
 - * Süslü F'nin sol tarafına bir şey yazılmazsa: bütün tablo üzerinde çalışır
 - * Süslü F'nin sol tarafına grupta niteliği yazılırsa: o niteliğe göre gruptandırıp her grup üzerinde fonksiyon çalışır
 - * Count NULL değerleri saymaz
 - * Count(*) veya Count(primary key) en güvenli kullanımdır
 - * Count Distinct: tekrarları ve NULL'ları çıkararak farklı değerleri sayar
- İki Tablo Operatörleri
 - Union ($\cup$) - Birleşim
 - * İki tablonun satırlarını birleştirir
 - * Kompatibilite şartı: aynı sayıda öz nitelik ve domenlerin aynı olması gerekir
 - * Şemanın (isimlerin) aynı olması şart değildir, domenlerin aynı olması yeterlidir
 - * Minimum sonuç: 1 satır (hepsi aynıysa), Maksimum: her iki tablonun satır sayıları toplamı
 - * İlişkisel cebirde tekrarlar çıkarılır, makinede (SQL'de) çıkarılmaz
 - Intersection ($\cap$) - Kesişim
 - * Hem R'de hem S'de bulunan ortak kayıtları verir

- * Kompatibilite gerekir
- * Minimum: 0 (boş tablo), Maksimum: küçük olan tablonun satır sayısı kadar
- * Join'dan farkı: Intersection tüm nitelikleri karşılaştırır, Join sadece belirtilen join condition üzerindeki nitelikleri karşılaştırır
- * Intersection'da NULL'lar aynı kabul edilir, Join'da NULL'lar aynı kabul edilmez
- * Join'da maksimum sonuç Cartesian çarpım kadardır, Intersection'da küçük tablo kadardır
- Set Difference (-) - Küme Farkı
 - * R'den S'deki kayıtları çıkarır
 - * Kompatibilite gerekir
 - * Commutative değildir: $R - S \neq S - R$
- Cartesian Product ($\times$) - Kartezen Çarpım
 - * İki tablonun bütün satırlarını birbirleriyle eşleştirir
 - * Minimum: bir tablo boşsa diğerinin satır sayısı kadar, Maksimum: R satır sayısı $\times$ S satır sayısı
 - * Performans açısından en yüklü işlemlerden biridir
- Theta Join ($\bowtie$) / Inner Join
 - * Cartesian çarpım + yatay seçim (select)
 - * Belirli bir yükleme (predicate) göre eşleşen satırları seçer
 - * SQL'de INNER JOIN olarak geçer
 - * Yükleme eşitlik ifadesi ise Equi Join olur
 - * Örnek: Employee $\bowtie$ Project.Code = Project.Code Project
 - * Smith A Venus $\rightarrow$ Smith A projesinde çalışıyor, projenin ismi Venus (mana ifade eder)
 - * Smith A B Mars $\rightarrow$ mana sıfır, hiçbir anlam ifade etmez
 - * Join, sistem için en problemlili ve en yüklü operatördür
- Natural Join ($\Join$)
 - * Predicate yazılmaz, default olarak iki tablodaki aynı isimli nitelikler üzerinden birleşim yapar
 - * Örnek: Employee $\Join$ Department $\rightarrow$ Department niteliği üzerinden join yapılır
- Semi Join ($\Join_{\text{semi}}$)
 - * Soldaki tablonun, sağdaki tablo ile eşleşen satırlarını seçer
 - * Sadece soldaki tablonun nitelikleri sonuçta yer alır
 - * Sağdaki tablo sadece seçim kriteri olarak kullanılır
 - * Commutative değildir: $R \Join_{\text{semi}} S \neq S \Join_{\text{semi}} R$
 - * Natural Join + Project olarak ifade edilebilir: πR 'nin nitelikleri($R \Join_{\text{semi}} S$)
 - * Örnek: Employee $\Join_{\text{semi}}$ Department $\rightarrow$ Departman kayıtlarında geçen departmanlarda çalışan işçileri bul
- Anti Join ($\Join_{\text{anti}}$)
 - * Soldaki tablonun, sağdaki tablo ile eşleşmeyen satırlarını seçer
 - * Semi Join'ın tam tersidir
 - * Commutative değildir
 - * Semi Join $\cup$ Anti Join = Soldaki tablo (Employee)
 - * Semi Join $\cap$ Anti Join = Boş küme
 - * Left Outer Join + NULL kontrolü + Project ile ifade edilebilir
- Left Outer Join ($\Join_{\text{left}}$)
 - * Normal join + soldaki tablonun eşleşmeyen satırları (sağ taraf NULL ile doldurulur)
 - * "Bölüm başkanı olmayan departmanları da göster" gibi sorgularda kullanılır
- Right Outer Join ($\Join_{\text{right}}$)
 - * Normal join + sağdaki tablonun eşleşmeyen satırları (sol taraf NULL ile doldurulur)
- Full Outer Join
 - * Her iki tablonun da eşleşmeyen satırları gösterilir
 - * $(A \Join_{\text{anti}} B) \cup (B \Join_{\text{anti}} A)$ ile ifade edilebilir
- Division ($\div$)
 - * İlişkisel cebirin en zor operatörü
 - * R'yi S'ye böler, sonuç Y niteliklerinden oluşan T tablosu
 - * Şart: S'nin nitelikleri R'nin niteliklerinin bir alt kümesi olmalı
 - * "Bütün" kelimesi geçen sorgularda kullanılır

- * Örnek: Smith'in çalıştığı BÜTÜN projelerde çalışan işçileri bul
- * İki negatif mana içerir → SQL'de iki tane NOT EXISTS ile gerçekleştirilir
- * Formal tanımı: T'deki değerler, R'de S'nin bütün değerleriyle kombinasyon halinde görünmeli
- Operatörlerin Özellikleri (Properties)
 - Commutative (Değişme) Özelliği
 - * Union, Intersection, Cartesian Product, Theta Join (Equi Join) commutative'dir
 - * Set Difference, Semi Join, Anti Join commutative değildir
 - * Division: R'yi S'ye bölmek tanımlı olabilir ama S'yi R'ye bölmek tanımsız olabilir
 - Associative (Birleşme) Özelliği
 - * Operatörlerin sıralanabilirliği ile ilgilidir
- Kümeleme Fonksiyonları ve Gruplama Detayları
 - Gruplama yapılmadığında: bütün tablo üzerinde işlem yapılır, sonuç tek satır
 - Gruplama yapıldığında: grup sayısı kadar satır çıkar
 - Gruplama yapıldıktan sonra sadece gruplama nitelikleri ve kümeleme fonksiyonu sonuçları kalır
 - Gruplama sonrası diğer niteliklere erişilemez (sık yapılan hata)
 - "For each" ifadesi gruplama gerektiğini gösterir
 - Gruplama niteliklerinin sırası sonucu değiştirmez (set noktasında aynı gruplar olur)
 - Sort (sıralama) niteliklerinin sırası sonucu değiştirir
- İlişkisel Cebir Ağacı
 - İşlem sırasını görsel olarak ifade eder
 - Alttan yukarıya doğru işlemler uygulanır
 - Sigma (yatay filtre) ve Pi (dikey filtre) yer değiştiremez
 - İki Sigma kendi arasında yer değiştirebilir
- Sorgu Yazma Stratejileri
 - Küçük dünyayı (veritabanı şemasını) tam anlamak kritik önemdedir
 - En çok yapılan hata: gereksiz join yapmak
 - Sorgu kurgusu: önce hangi bilgi nerede, hangi tablolarda bulunur onu belirle
 - Aynı tabloda bilgi bulunuyorsa join yapmaya gerek yoktur
 - Geçici (temporary) tablolar ile adım adım sorgu oluşturma
 - View tanımlama ile geçici tablolar SQL'de gerçekleştirilebilir
- SQL'e Geçiş Notları
 - SELECT * FROM R, S → Cartesian Product ($R \times S$)
 - SELECT * FROM R JOIN S ON condition → Theta Join
 - SQL'de JOIN kullanımı (SQL 98/2000 standardı): INNER JOIN ile join condition yazılır
 - Cartesian product her zaman WHERE condition ile kullanılmalı

Hocanın Özellikle Vurguladığı Kısımlar

- İlişkisel cebir set esaslıdır; tekrarlar (duplicate) otomatik çıkarılır, makinede (SQL'de) ise çıkarılmaz
- Join operasyonu sistem için en yüklü ve en problemli operatördür
- Division operatörü ilişkisel cebirin en zor noktasıdır; "bütün" kelimesi division gerektiğini gösterir
- Kümeleme fonksiyonlarında NULL sayılmaz; Count(*) veya Count(primary key) en güvenli kullanımdır
- Gruplama yapıldıktan sonra gruplama nitelikleri dışındaki niteliklere erişilemez — en sık yapılan hatalardan biridir
- Küçük dünyayı (şemayı) tam anlamadan sorgu yazmak gereksiz join'lere yol açar
- Gereksiz join yapmak sınavda puan kırma sebebidir; doğru sorgu olsa bile puanın yarısı gider
- Sigma ve Pi operatörlerinin sırası önemlidir; yer değiştiremezler
- Sort işleminde nitelik sırası sonucu değiştirir; gruplama nitelik sırası sonucu değiştirmez
- Anti Join, Left Outer Join ile ifade edilebilir (NOT NULL kontrolü ile)
- Semi Join + Anti Join = Soldaki tablo; Semi Join $\cap$ Anti Join = Boş küme

Kısa Tekrar Notları

- σ (sigma) = yatay seçim (satır filtreleme), yüklem ile
- π (pi) = dikey seçim (sütun seçme), tekrarlar otomatik çıkarılır
- $\Join$ (join) = kartezen çarpım + yükleme filtresi
- Natural Join = aynı isimli nitelikler üzerinden otomatik join
- Semi Join ($\bowtie$) = soldaki tablonun eşleşen satırları
- Anti Join = soldaki tablonun eşleşmeyen satırları
- Left Outer Join = join + soldaki eşleşmeyenler (sağ taraf NULL)
- Division ($\div$) = “bütün” kelimesi $\rightarrow$ en zor operatör, iki NOT EXISTS ile SQL’de
- Union = birleşim (kompatibilite gerekir, tekrarlar çıkar)
- Intersection = kesişim (NULL’lar aynı kabul edilir)
- Set Difference = küme farkı (commutative değil)
- Cartesian Product = tüm kombinasyonlar ($R \times S$ satır sayısı)
- Aggregate fonksiyonlar: count, sum, avg, min, max
- Gruplama yoksa $\rightarrow$ tek satır sonuç; gruplama varsa $\rightarrow$ grup sayısı kadar satır
- Count NULL saymaz, Count(*) tüm satırları sayar
- Rename (ρ) = nitelik/tablo ismi değiştirme, özellikle Natural Join’da kullanılır
- Extend = şemaya yeni nitelik ekleme veya hesaplama
- Sort = sıralama, outmost operatördür (en son uygulanır)
- Complete set: $\{\sigma, \pi, \cup, -, \times, \rho\}$ — tüm ilişkisel cebir işlemleri bu setle ifade edilebilir
- Join, Division bu temel setten türetilebilir

Detaylı Açıklamalar

- **İlişkisel Cebir ve SQL İlişkisi:** İlişkisel cebir, SQL sorgularının arka planında dönen işlemleri tanımlar. SQL son kullanıcıya yönelik bir sorgulama diliyken, ilişkisel cebir makineye yakındır ve verinin nasıl erişileceğini ifade eder. İlişkisel cebir altyapısını anlamak, SQL yazma noktasında büyük kolaylık sağlar. Ancak SQL’de gramere (syntax) hakim olmak ve kullanılan veri tabanına özgü syntax farklılıklarını bilmek gerekir.
- **Select ve Project Farkı:** Select (σ) tablodaki satırları bir yükleme (AND, OR, NOT ifadeleri) ile filtreler — yatay seçim. Project (π) ise sütunları seçer — dikey seçim. Project’te yükleme yoktur. İlişkisel cebir set esaslı olduğundan project sonucunda tekrarlar otomatik çıkarılır; örneğin A ve C nitelikleri seçildiğinde duplicate satırlar tek satıra düşer. Gerçek veri tabanında (SQL’de) ise tekrarlar çıkarılmaz.
- **Join Operasyonunun Mantığı:** Join işlemi, arka planda kartezen çarpım yapıp ardından bir yükleme (predicate) ile filtreleme yapar. Bu yükleme eşitlik ifadesi ise equi join olur. Join’da amaç mana ifade eden satırları seçmektir. Örneğin Employee ve Project tablolarını Project.Code üzerinden join yaptığımızda, Smith’in A projesinde çalıştığı ve projenin isminin Venus olduğu bilgisi mana ifade ederken; Smith A B Mars gibi eşleşmeler mana sıfırdır. Bu nedenle join, sistem için en yüklü operatördür.
- **Outer Join Çeşitleri ve Kullanım Alanları:** Left Outer Join, normal join sonucuna ek olarak soldaki tablonun eşleşmeyen satırlarını da gösterir (sağ taraf NULL). Bu, “bölüm başkanı olmayan departmanları da göster” gibi sorgularda kullanılır. Right Outer Join tam tersidir. Full Outer Join her iki taraftaki eşleşmeyenleri de gösterir. Anti Join, Left Outer Join yapıp B tarafında NULL olanları seçerek gerçekleştirilebilir.
- **Division Operatörünün Zorluğu ve Kullanımı:** Division, R tablosunu S tablosuna böler. Sonuç, S’nin nitelikleri dışında kalan R’nin niteliklerinden oluşur. Şart: S’nin nitelikleri R’nin niteliklerinin bir alt kümesi olmalı. Division’ın en zor yanı “bütün” kelimesini içeren sorgularda kullanılmasıdır. Örneğin “Smith’in çalıştığı BÜTÜN projelerde çalışan işçileri bul” sorgusunda, herhangi birinde değil hepsinde çalışma şartı vardır. Division iki negatif mana içerir ve SQL’de iki NOT EXISTS ile gerçekleştirilir.
- **Kümeleme Fonksiyonları ve Gruplama:** Kümeleme fonksiyonları (count, sum, avg, min, max) bir tablo veya grup üzerinde istatistiksel işlem yapar. Gruplama yoksa bütün tablo üzerinde çalışılır ve sonuç tek satırdır. Gruplama varsa (süslü F’nin sol tarafında gruplama niteliği), her grup için ayrı ayrı işlem yapılır ve sonuç grup sayısı kadar satır olur. Kritik nokta: gruplama yapıldıktan sonra sadece gruplama nitelikleri ve kümeleme

fonksiyonu sonuçları kalır; diğer niteliklere erişilemez. Bu, SQL'de en sık yapılan hatalardan biridir.

- **Count ve NULL İlişkisi:** Count fonksiyonu NULL değerleri saymaz. Bu nedenle “bölümü olan öğrenci sayısını” bulmak için Count(Major ID) kullanılabilir çünkü Major ID'si NULL olan öğrenciler (bölümü olmayanlar) sayılmaz. En güvenli kullanım Count(*) veya Count(primary key) şeklindedir. Count Distinct ise hem NULL'ları hem tekrarları çıkarak farklı değer sayısını verir.
- **Sorgu Kurgulama Stratejisi:** Sorgu yazarken önce küçük dünyayı (veritabanı şemasını, tabloları, yabancı anahtarları) tam anlamak gerekir. En çok yapılan hata, sorguyu tam anlamadan gereksiz join yapmaktır. Eğer aranan bilgi tek tabloda bulunuyorsa join yapılmamalıdır. Sorgu adım adım kurgulanmalı: önce hangi tablolarda bilgi var, hangi filtreler gerekli, grupta gerekiyor mu, sıralama gerekiyor mu. Karmaşık sorgularda geçici tablolar (temporary tables) veya view'lar kullanılabilir.
- **Operatör Sırası ve İlişkisel Cebir Ağacı:** Operatörlerin uygulanma sırası önemlidir. Sigma (yatay filtre) ve Pi (dikey filtre) yer değiştiremez — önce Pi uygulanırsa filtreleme için gerekli nitelikler kaybolabilir. İki Sigma kendi arasında yer değiştirebilir. Sort işlemi çoğu zaman outmost (en dış) operatördür, yani diğer işlemler bittikten sonra uygulanır. Sort niteliklerinin sırası sonucu değiştirir: önce A'ya göre sonra B'ye göre sıralamak ile önce B'ye göre sonra A'ya göre sıralamak farklı sonuçlar üretir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.